

Deep Learning Fundamentals - Assignment 1

Gauranga Das
University of Adelaide
a1910652@adelaide.edu.au

1. Introduction

For this assignment, the task is to use the Perceptron [2] learning algorithm to perform binary classification on the Pima Indians Diabetes Database [1] to predict whether or not a patient has diabetes using some medical measurements. The dataset has 9 columns in total, out of which 8 are input features and the 'Outcome' column contains binary targets to be predicted. The Perceptron learning algorithm is very simple and easy to understand. However, it has limitations due to its linear nature.

For this task, the Perceptron learning algorithm is implemented and it is then evaluated. Then, some modifications are made to the baseline Perceptron model, to see if the results could be further improved.

2. Method

Our goal is to perform binary classification on the Pima Indians Diabetes Database. To achieve this, the Perceptron learning algorithm is used.

2.1. Model

The Perceptron model is a linear classifier, as it learns a linear decision boundary to separate two classes. Given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ consisting of input-output pairs such that $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$. The Perceptron model has parameters $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ and the output of the Perceptron model is $g(\mathbf{x}_i; \mathbf{w})$:

$$g(\mathbf{x}_i; \mathbf{w}) = \text{sign}(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)$$

where $\text{sign}(\cdot)$ is the signum function. For simplicity, the bias is removed and the output of the Perceptron becomes:

$$g(\mathbf{x}_i; \mathbf{w}) = \text{sign}(\langle \mathbf{w}, \mathbf{x}_i \rangle)$$

To optimize the model parameters $\mathbf{w}$, the Perceptron loss function $l_{\text{pern}}(\mathbf{x}_i, y_i, \mathbf{w})$ is used which calculates the loss for a single input-output pair:

$$l_{\text{pern}}(\mathbf{x}_i, y_i, \mathbf{w}) = \max(0, -y_i \langle \mathbf{w}, \mathbf{x}_i \rangle)$$

Therefore, we can define the empirical risk $R_n(\mathbf{w}, l_{\text{pern}})$ which is the average loss over the training data:

$$R_n(\mathbf{w}, l_{\text{pern}}) = \frac{1}{n} \sum_{i=1}^n l_{\text{pern}}(\mathbf{x}_i, y_i, \mathbf{w})$$

We now have to find $\mathbf{w}$ that minimizes the empirical risk. To minimize the risk, we could calculate the gradient of the

risk function. However, it is not differentiable because of the signum function. Therefore, we instead calculate the sub-gradient which provides us a direction that can be used to minimize the risk:

$$\frac{\partial R_n(\mathbf{w}, l_{\text{pern}})}{\partial \mathbf{w}} = -\frac{1}{n} \sum_{i=1}^n (y_i \mathbf{x}_i 1_{\{y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \leq 0\}})$$

This can then be used to update the parameter $\mathbf{w}$ as:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{i=1}^n (y_i \mathbf{x}_i 1_{\{y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \leq 0\}})$$

where η is the step size or learning rate.

Algorithm 1 Perceptron Algorithm

Input: Training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, step size η , number of iterations T

Initialize: $\mathbf{w}_1 = \mathbf{0}$

for $t = 1$ to T **do**

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta \sum_{i=1}^n (y_i \mathbf{x}_i 1_{\{y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \leq 0\}})$$

end for

Output: $\mathbf{w}^* = \mathbf{w}_T$

The Perceptron model can then be used to predict the class of a new input $\mathbf{x}_i$:

$$y_{\text{pred}} = \text{sign}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle)$$

2.2. Limitations

Unlike most popular algorithms, the Perceptron model is relatively much simpler and is therefore, very interpretable. However, it also has major limitations. The Perceptron model is a linear classifier, which makes it suitable for classifying data that is linearly separable. But, if the data is not linearly separable, the algorithm fails to give good results. Since, it has no hidden layers with non-linear activation functions, it cannot learn complex patterns in the data. The model is also very sensitive to outliers. This is because it tries to find a linear decision boundary that will separate all the training examples, including the outliers. The outliers can also cause the weights to continuously adjusting, preventing it from converging. Also, the Perceptron model does not provide any probability estimate of a class.

2.3. Improvements

To improve upon the baseline Perceptron model, some modifications have been made. The original model has

Table 1. Performance metrics (in %) of the baseline model and improved models on the test dataset

Model	Accuracy	Precision	Recall	F5
Baseline	66.38	100.00	2.5	2.6
LR Scheduling	71.55	59.46	55.00	55.16
LR Scheduling + L2 Regularization	75.86	65.79	62.50	62.62
LR Scheduling + Bias	74.14	63.16	60.00	60.12

a constant learning-rate, which can be problematic if the learning rate is too high. When updating weights, this can cause the weights to oscillate around the minimum of the empirical risk function without actually reaching the minimum or simply overshoot the minimum and cause further divergence. Therefore, learning-rate scheduling [7] is used which decreases the learning rate η by a factor of 0.1 every 200 iterations.

L2 regularization [6] is also added with learning rate scheduling to see if that leads to further improvements. Adding L2 regularization will penalize larger weights and therefore prevent overfitting which could lead to improvement in model performance. When L2 regularization is added the empirical risk function becomes:

$$R_n(\mathbf{w}, l_{pern}) = \frac{1}{n} \sum_{i=1}^n l_{pern}(\mathbf{x}_i, y_i, \mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

where λ is the L2 regularization strength. The sub-gradient of the risk function becomes:

$$\frac{\partial R_n(\mathbf{w}, l_{pern})}{\partial \mathbf{w}} = -\frac{1}{n} \sum_{i=1}^n (y_i \mathbf{x}_i 1_{\{y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle) \leq 0\}}) + \lambda \mathbf{w}$$

The weight parameter will now be updated as:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (\sum_{i=1}^n (y_i \mathbf{x}_i 1_{\{y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle) \leq 0\}}) - \lambda \mathbf{w})$$

The L2 regularization strength λ is set to 0.5.

The Perceptron model used so far has no bias term. So, a bias term is added with learning-rate scheduling. Without the bias term, the decision boundary of the Perceptron model always passes through origin. Adding the bias allows the decision boundary to be shifted away from the origin, thus making the model more flexible and better able to classify the data points. To add the bias term, we simply add an extra term to our weight vector (which will be the bias) and to take into account the extra term, we augment our input vector with a 1 which will be multiplied with the bias. Thus, our model output becomes:

$$g(\tilde{\mathbf{x}}_i; \tilde{\mathbf{w}}) = \text{sign}(\langle \tilde{\mathbf{w}}, \tilde{\mathbf{x}}_i \rangle)$$

where $\tilde{\mathbf{w}} \in \mathbb{R}^{d+1}$ is the model parameter and $\tilde{\mathbf{x}}_i = [\mathbf{x}_i; 1]$. This ensures that in the formulas for the Perceptron loss function, the empirical risk function and the sub-gradient remains we only need to replace $\mathbf{x}_i$ with $\tilde{\mathbf{x}}_i$ and $\mathbf{w}$ with $\tilde{\mathbf{w}}$. Our update rule now becomes:

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta \sum_{i=1}^n (y_i \tilde{\mathbf{x}}_i 1_{\{y_i(\langle \tilde{\mathbf{w}}, \tilde{\mathbf{x}}_i \rangle) \leq 0\}})$$

3. Experiments

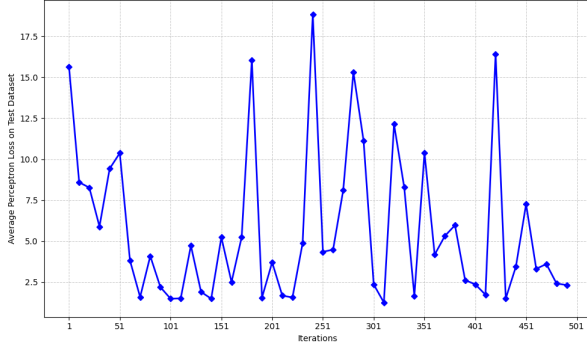
The Perceptron learning algorithm was trained and evaluated on the Pima Indians Diabetes Database. The experiments illustrate that even though the Perceptron model is a

linear classifier, it can achieve good performance on small datasets.

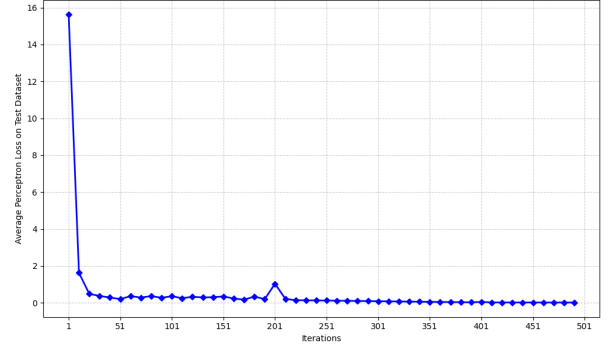
Data Preprocessing. The original Pima Indians Diabetes Database is used from Kaggle. Each target label is either a 0 or 1, with 1 indicating that the disease is present. The Perceptron model implemented here expects the outputs to be either +1 or -1. Therefore, the target labels with values 0 are changed to -1. Also, the input features are scaled using min-max normalization [5] with each column being scaled in the range $[-1, 1]$. Finally, the entire dataset is split into training and test sets.

Evaluation Methods. During training, the Perceptron loss function was used to calculate the average loss of a model on the test set after every 10 iterations. This is used to plot a loss curve for each model. After training the model on the training set, it was evaluated on the test set to measure its performance on unseen data. Since, it is a binary classification problem the metrics used for evaluation were accuracy, precision, recall and F5 score [3, 4]. In medical settings, accuracy alone is not important and other metrics should also be considered. False negatives is more important than false positives. This is because if a patient with diabetes is falsely predicted to not have diabetes (false negative), this will lead to the patient not receiving treatment for the disease which can be fatal for them. However, if a patient who doesn't have diabetes is falsely predicted to have diabetes (false positive), this will definitely cause problems for the patient, however this is still much less worse than a false negative. Therefore, for our purposes we will judge the model based on accuracy, precision and recall but, with more importance given to the recall score since maximizing the recall score means minimizing the number of false negatives. Therefore, the primary metric used for choosing the best model will be the F5 score since this metric uses both precision and recall but, it gives recall 5 times more importance than precision.

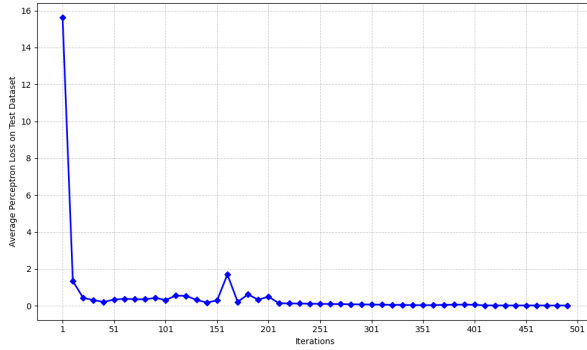
Results. Table 1 compares the performance of the baseline model and the effects of adding further improvements. We can see that while the baseline model achieves good accuracy and precision, its recall was poor thus leading to a poor F5 score. This implies that when a patient actually has the disease, the baseline model was unable to predict that the patient actually has the disease with good accuracy. In real-life this can be dangerous since, such a model will predict



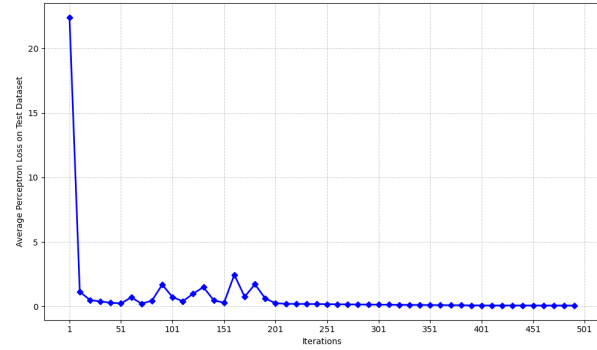
(a) Baseline



(b) LR Scheduling



(c) LR Scheduling + L2 Regularization



(d) LR Scheduling + Bias

Figure 1. Loss curves on the test dataset

a lot of patients don't have diabetes when they actually do have the disease. This will lead to a lot of diabetic patients not receiving treatments which could lead to the disease becoming worse and potentially causing their death. Figure 1 shows that the baseline model's loss is fluctuating a lot, implying that the model could not converge properly. This could be due to a high learning-rate, which is preventing the model from reaching the minimum of the risk function. To improve upon this baseline model, learning-rate scheduling was added. It can be observed that this leads to improved accuracy and significant increase in recall. Even though, this model has lower precision score than the baseline model, it has both good recall and precision scores which is evident by its higher F5 score. From Figure 1, we can observe that the loss decreases constantly and is no longer fluctuating, implying that the model has converged. Therefore, it is much more preferable than the baseline model. When L2 regularization is added to learning-rate scheduling, the accuracy, precision and recall increases leading to a higher F5 score. This implies that adding L2 regularization made the model better since, larger weights are penalized which reduces overfitting and improves the model's performance on unseen data. From Figure 1, we can observe that the loss curve with learning rate scheduling and L2 regularization is

similar to the previous model and that it has converged due to decreasing loss. Finally, when the bias term is added to learning-rate scheduling, we get better accuracy, precision and recall compared to only adding learning-rate scheduling. Increasing the bias makes the model more flexible, which explains the improvement in performance.

4. Code

GITHUB LINK

5. Conclusions

In this paper, the Perceptron algorithm was applied to the Pima Indians Diabetes Database to predict whether a patient has diabetes or not. The baseline model did not perform well on the dataset. Several improvements were made to the baseline model, such as learning rate scheduling, adding L2 regularization, and including a bias term. These modifications did improve the model's performance significantly, suggesting that they were effective in improving the baseline model. However, even with these improvements, the model's overall performance remained sub-par. Such models would not be very useful in real-world medical settings.

The experiments demonstrate that the Perceptron model can achieve decent performance on small datasets that are

linearly separable. But, they are not powerful enough to achieve excellent performance and could struggle with large datasets which have data that are not linearly separable. The primary reason for this is due to the linear nature of the Perceptron model.

For further improvements, multi-layered perceptrons (MLP) with non-linear activation functions could be used to further enhance the performance. MLPs can model complex non-linear relationships, which allows them to learn more complex patterns in the data.

References

- [1] UCI Machine Learning Repository. Pima Indians Diabetes Dataset, 2016. Kaggle.
- [2] Frank Rosenblatt. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6):386–408, 1958.
- [3] scikit-learn. `fbeta_score`. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.fbeta_score.html. (accessed 19 Sep. 2024).
- [4] scikit-learn. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html. (accessed 13 Sep. 2024).
- [5] scikit-learn. `MinMaxScaler`. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>. (accessed 11 Sep. 2024).
- [6] Steph. Gradient descent: L2 norm regularization. <https://math.stackexchange.com/questions/1652661/gradient-descent-l2-norm-regularization>, 2022. (accessed 13 Sep. 2024).
- [7] Ashton Zang, Zachary Lipton, Mu Li, and Alexander Smola. Learning Rate Scheduling. https://d2l.ai/chapter_optimization/lr-scheduler.html. Dive into Deep Learning.